

VNUHCM - University of Science
Faculty of Information Technology

CS486 – Lecture 11

Query Optimization

Le Thi Nhan

ltnhan@fit.hcmus.edu.vn

Content

- [illegible]

Example

- R(A, B, C)
 - S(C, D, E)
- SELECT B, D
FROM R, S
WHERE R.A='c' AND S.E=2 AND R.C=S.C

R

A	B	C
a	1	10
b	1	10
c	2	10
d	2	10
e	3	10
...	...	...

S

C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3
...	...	...

Answer

B	D
2	x

- How do we execute query?

Idea 1

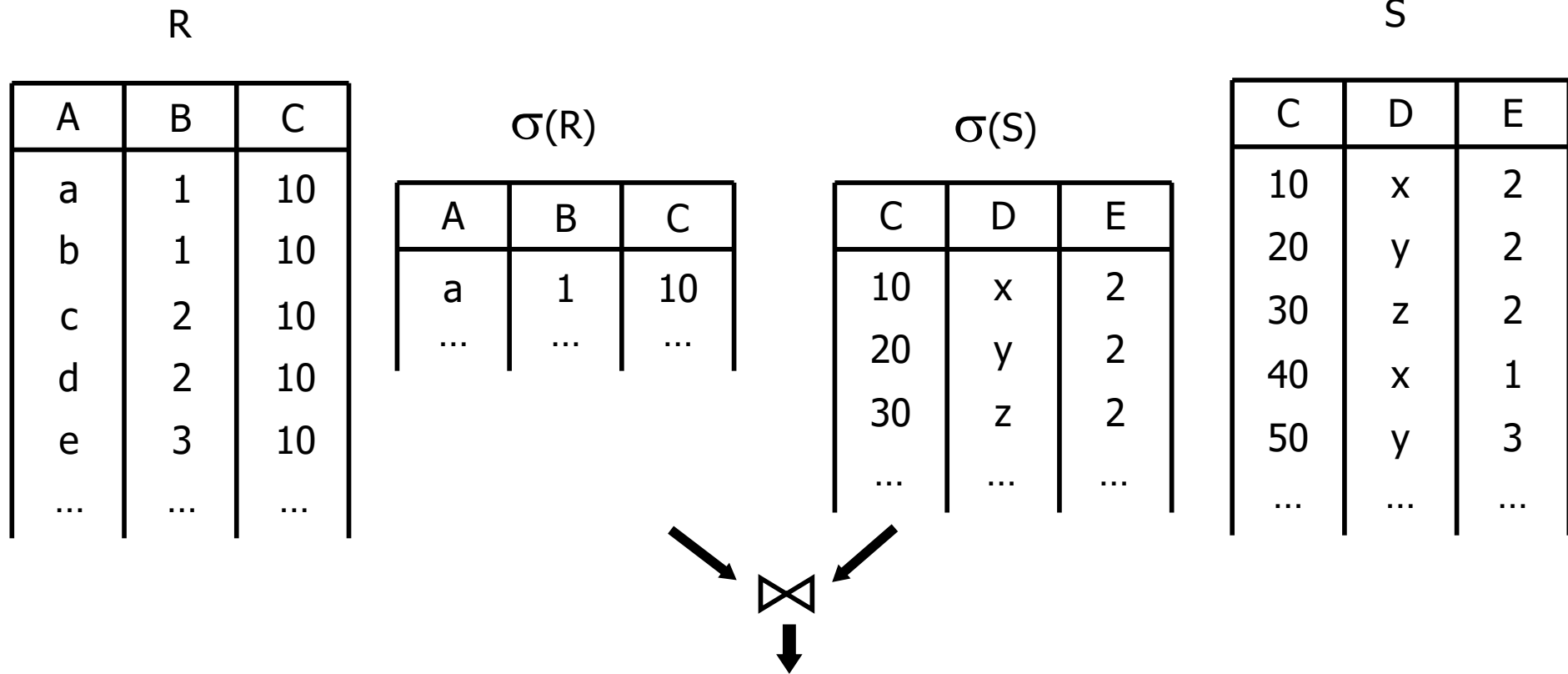
- Do Cartesian product
- Select tuples
- Do projection

R × S

A	B	C	C	D	E
a	1	10	10	x	2
a	1	10	20	y	2
⋮					
c	2	10	10	x	2
c	2	10	20	y	2
c	2	10	30	z	2
⋮					

$$\pi_{B,D} [\sigma_{R.A='c' \wedge S.E=2 \wedge R.C = S.C} (R \times S)]$$

Idea 2

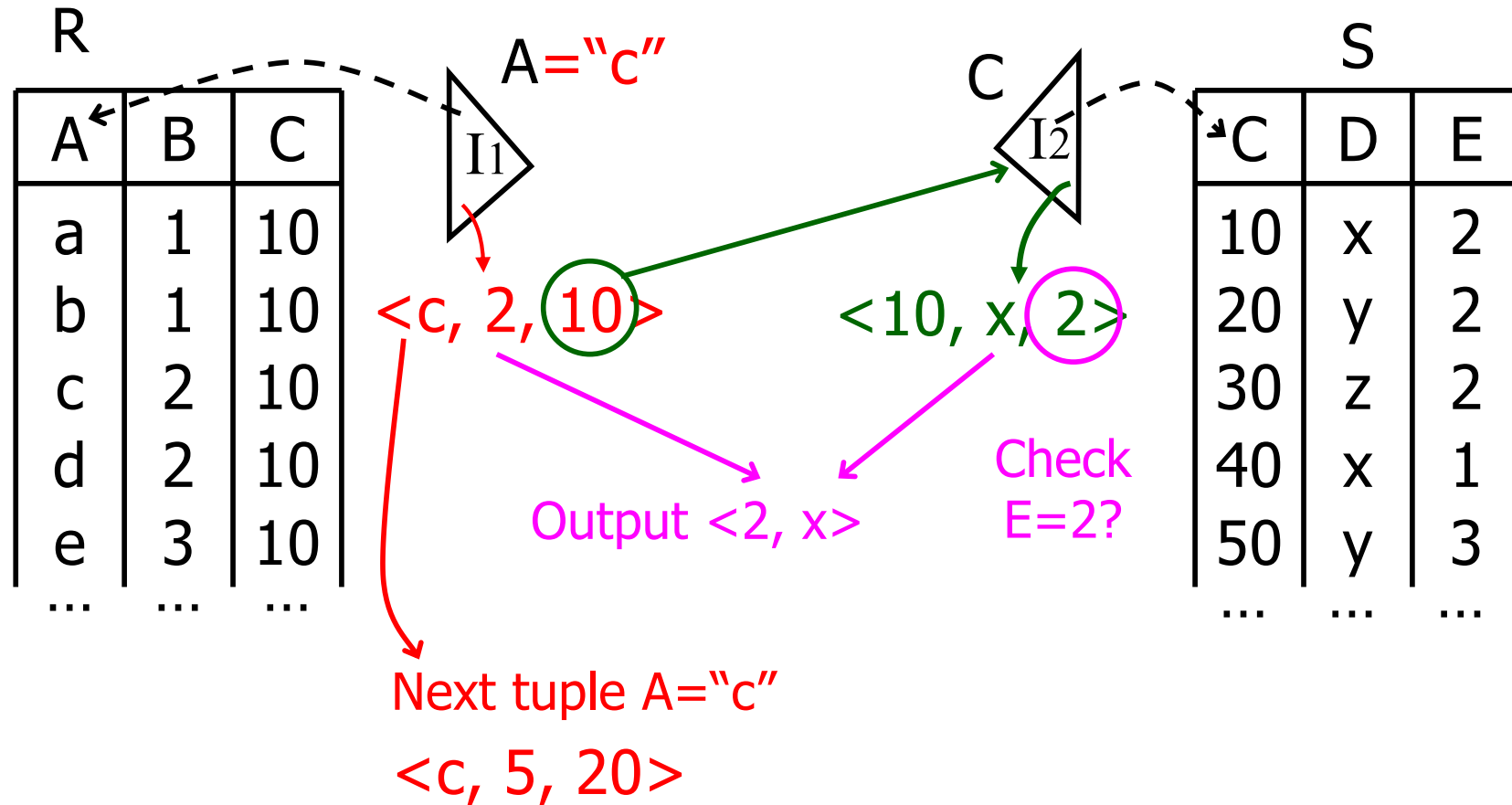


$$\pi_{B,D} [\sigma_{R.A='c'} (R) \bowtie \sigma_{S.E=2} (S)]$$

Idea 3

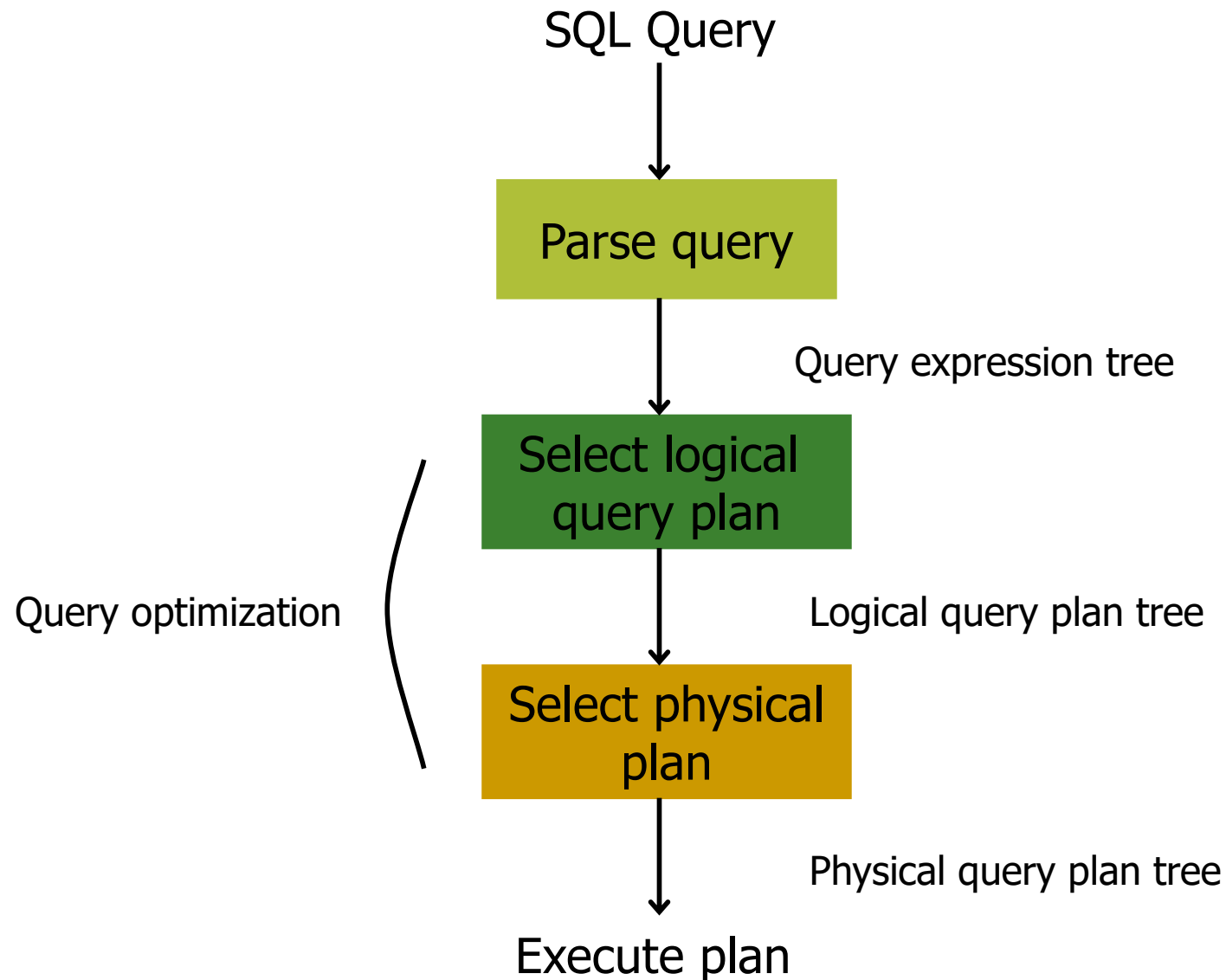
- Use R.A and S.C indexes
 - Use R.A index to select R tuples with R.A = “c”
 - For each R.C value found, use S.C index to find matching tuples
 - Eliminate S tuples S.E $\neq$ 2
 - Join matching R, S tuples
 - Project B, D attributes and place in result

Idea 3



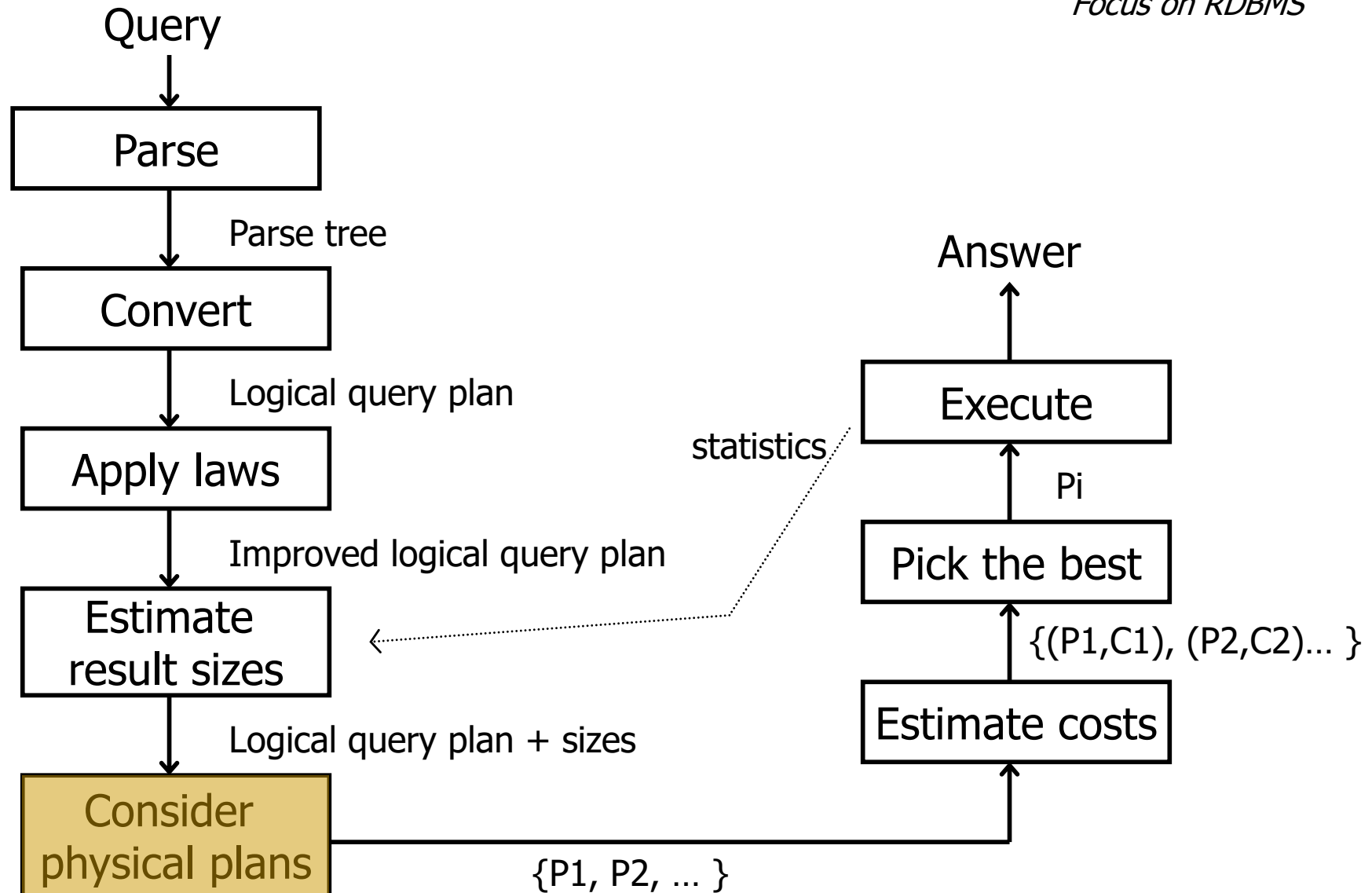
Query Processing

Focus on RDBMS

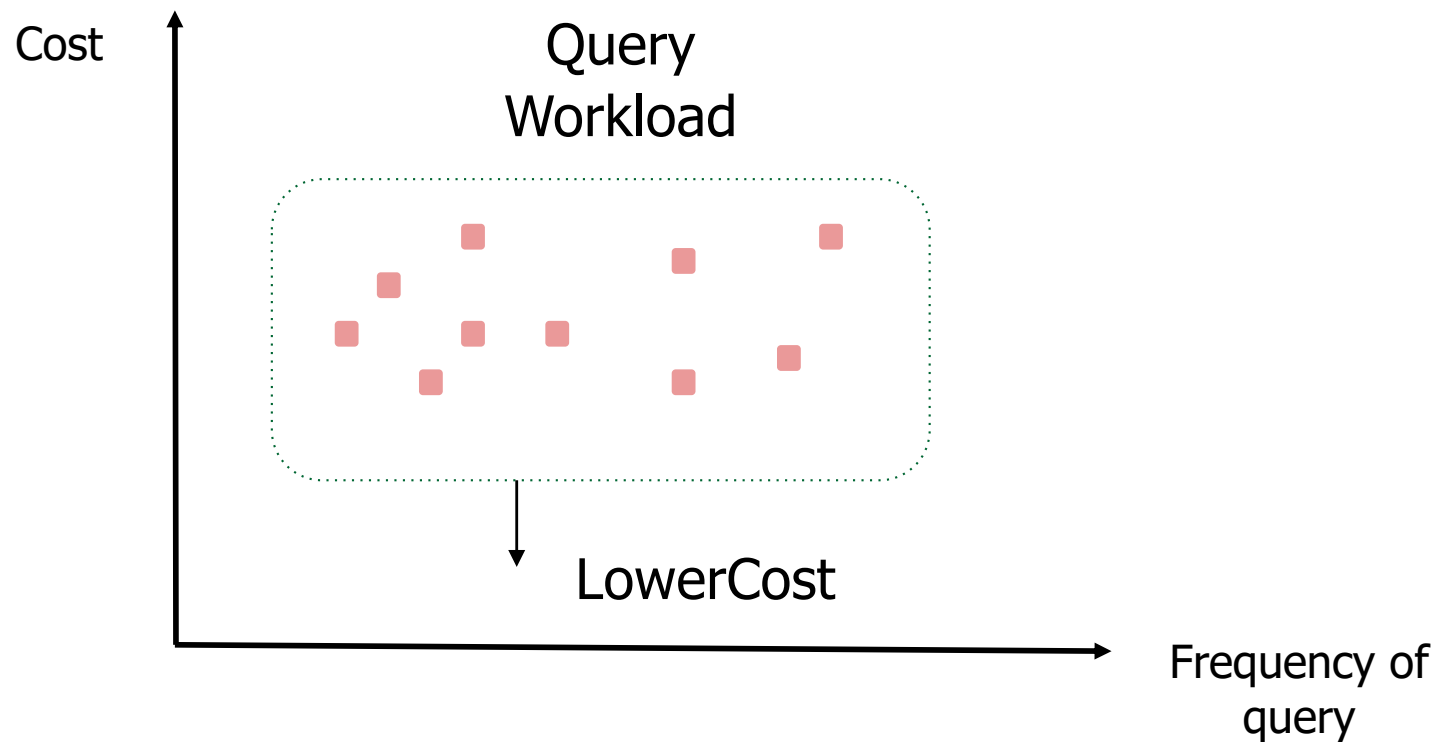


Query processing

Focus on RDBMS



Workload



Workload = <Query, Frequency of query>

Workload

Q1

```
SELECT  ESSN, COUNT(*) as Num_Prj
FROM    WORKS_ON
WHERE   Hours=20.5
GROUP BY ESSN
ORDER BY Num_Prj DESC
```

Q2

```
SELECT  SSN, FName
FROM    EMPLOYEE, DEPT_LOCATIONS
WHERE   DLocation= 'Houston'
AND     DNo=DNumber
```

Q3

```
SELECT  DName, AVG(Salary) AS Avg_Salary
FROM    EMPLOYEE, DEPARTMENT
WHERE   DNo=DNumber
GROUP BY DNo, DName
HAVING  AVG(Salary) > 20000
```

- Access the same data
- Compete for I/O resources
- Share physical structures
(indexes, materialized
views, caches)

SFW Query

■ Q1 `SELECT` `ESSN, COUNT(*)` as Num_Prj
 `FROM` `WORKS_ON`
 `WHERE` `Hours=20.5`
 `GROUP BY` `ESSN`
 `ORDER BY` `Num_Prj DESC`

- **WHERE:** Quickly locate the rows that match the condition using an index on Hours
- **GROUP BY:** Even with an index, still need to sort or hash-partition the rows to group
- **ORDER BY:** Sort the result by number of projects in descending order

SFW Query

■ Q3 **SELECT** DName, **AVG**(Salary) **AS** Avg_Salary
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNo=DNumber
 GROUP BY DNo, DName
 HAVING **AVG**(Salary) > 20000

- **WHERE:** Join Employee with Department to find rows by the same department number
 - Evaluate the costs of different JOIN algorithms, based on the size of the data and the presence of indexes
- **GROUP BY:** Group rows by department number and name, then compute the average salary for each group
 - Evaluate the cost among GROUP BY algorithms
- **HAVING:** Filter the groups with the average salary more than 20000

Query Optimizer

- Analyze
 - Each query
 - Available indexes
 - Statistics
 - Other information of data

- Generate an execution plan (the most efficient way to execute)
 - Select the most appropriate indexes to use
 - Reorder operations in the query
 - Make other adjustmentsto improve its performance

Content

- Overview
- **Join algorithms**

How to optimize JOIN queries

■ Join algorithms

- Nested Loop Join (NLJ)
- Block Nested Loop Join (BNLJ)
- Index Nested Loop Join (INLJ)
- Sort-Merge Join (SMJ)
- HashPartition Join (HPJ)

■ Goal

- Exploit memory efficiently by **reducing** the amount of data that needs to be compared during the join process

■ Cost : Disk IO

- Ignore CPU costs, timing, double buffering requirements

Note

■ IO cost

- Based on the number of **pages** loaded, not the number of tuples
 - Small system : 1 page = 8KB = one IO
 - Big system : 1 page = 64MB = one IO

■ Given a relation R , let

- $T(R)$ = # of tuples in R
- $P(R)$ = # of pages in R
- B = # memory blocks/pages available

Nested Loop Join (NLJ)

Cost

$$P(R) + T(R) \times P(S) + OUT$$

Compute $R \bowtie S$ on A :

for r in R

for s in S

if $r[A] == s[A]$

yield (r, s)

Loop over the tuples in R

For every tuple in R , loop over all the tuples in S

Check against join condition (check in the if statement)

Write out to page, then when page is full, write to disk

Size of OUT = how many rows match

What if R ("outer") and S ("inner") switched?

$$P(S) + T(S) \times P(R) + OUT$$

Need to know which relation is smaller

Block Nested Loop Join

Given : $B + 1$ buffer space

Idea : Use $B - 1$ pages for Employee, 1 page each for Works_On and Output

1 billion people
1000 tuples/page
(million pages)

432-567-789	
.....	
.....	
.....	
.....	
134-562-184	
.....	
.....	
.....	
613-416-452	

Employee table
(row store)

.....	
134-562-184	
.....	
613-416-452	
.....	
.....	
.....	
.....	
.....	
432-567-789	

Works_On table
(row store)



Output

Steps : Repeat till done

Read $B - 1$ pages from Employee into buffer

Read 1 page from Works_On

Partial join into 1 output page

Block Nested Loop Join (BNLJ)

Given $B + 1$ pages of memory

Cost

$$P(R) + \frac{P(R)}{B - 1} P(S) + OUT$$

Compute $R \bowtie S$ on A :

for each $B - 1$ pages pr of R

for page ps of S

for each tuple r in pr

for each tuple s in ps

if $r[A] == s[A]$

yield (r, s)

Load in $B - 1$ pages of R at a time

For each $(B - 1)$ page segment of R , load each page of S

Check against join condition

Write out

NLJ vs. BNLJ

■ In BNLJ

- Loading larger chunks of R
- Minimize the number of full disk reads of S
 - Only read all of S from disk for every $(B - 1)$ page segment of R
 - Still full cross product, but more done only in memory

$$P(R) + T(R) \times P(S) + OUT$$



$$P(R) + \frac{P(R)}{B - 1} P(S) + OUT$$

BNLJ is faster by roughly $\frac{(B-1)T(R)}{P(R)}$

Example

- $P(R) = 1000$, $P(S) = 500$
- 100 tuples/page
- $T(R) = 1000 \times 100$, $T(S) = 500 \times 100$

	$B + 1 = 100$ ($B = 99$)	$B + 1 = 20$ ($B = 19$)
NLJ $P(R) + T(R) \times P(S) + OUT$	$(1000 + 1000 \times 100 \times 500 + OUT)$ $\Rightarrow IO \approx 50,001,000 + OUT$	$(1000 + 1000 \times 100 \times 500 + OUT)$ $\Rightarrow IO \approx 50,001,000 + OUT$
BNLJ $P(S) + \frac{P(S)}{B - 1} P(R) + OUT$	$500 + \frac{500}{99 - 1} 1000 + OUT$ $\Rightarrow IO \approx 5.6K + OUT$	$500 + \frac{500}{19 - 1} 1000 + OUT$ $\Rightarrow IO \approx 28.2K + OUT$

Smarter Joins

- Joins computing the *full cross-product* have a quadratic term
 - $P(R) + T(R) \times P(S) + OUT$
 - $P(R) + \frac{P(R)}{B-1} P(S) + OUT$
- Other joins could gain from quadratic to nearly linear by taking advantage of structure (equijoin only)
 - $P(R) + P(S) + OUT$

Smarter Joins

1 billion people
1000 tuples/page
(million pages)

The diagram shows a table with two columns. The first column contains several rows of data, with some rows represented by dots indicating they are not shown. A green arrow points to the first row, which contains the SSN 432-567-789. A red dot is placed next to the SSN 134-562-184 in the second column. The table is divided into pages by dashed blue lines.

432-567-789	
.....	
.....	
.....	
.....	
134-562-184	
.....	
.....	
.....	
613-416-452	

Employee table
(row store)

The diagram shows a table with two columns. The first column contains several rows of data, with some rows represented by dots indicating they are not shown. A green arrow points to the first row, which contains the SSN 134-562-184. A red dot is placed next to the SSN 613-416-452 in the second column. The table is divided into pages by dashed blue lines.

.....	
134-562-184	
.....	
613-416-452	
.....	
.....	
.....	
.....	
.....	
432-567-789	

Works_On table
(row store)

The extra work BNLJ
is doing to join for
432-567-789

Index Nested Loop Join (JNLJ)

Compute $R \bowtie S$ on A :
Given index idx on $S.A$
for r in R
 s in $idx(r[A])$
 yield (r, s)

Cost

$$P(R) + T(R) \times L + OUT$$

L is the IO cost to access each distinct value in index (eg. B+ tree)

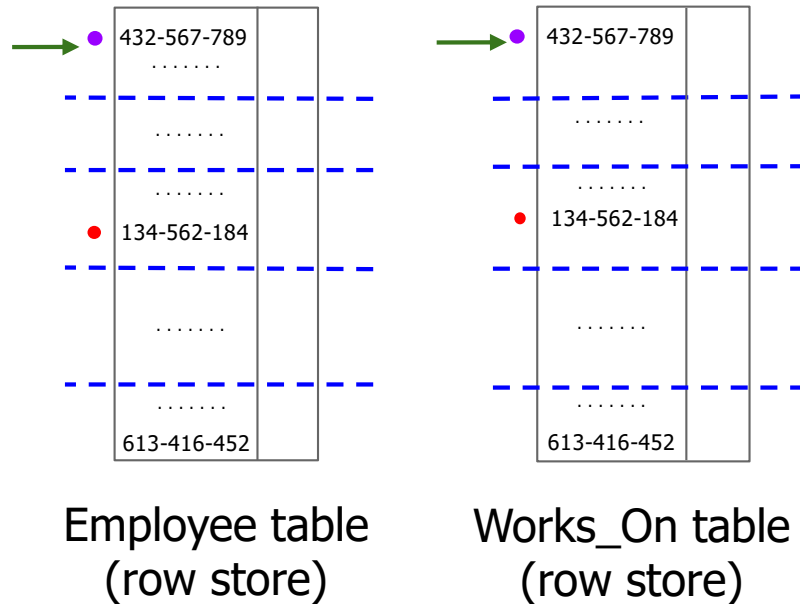
L is usually small, eg. 3-5

Note

- Given a relation R , let
 - $T(R)$ = # of tuples in R
 - $P(R)$ = # of pages in R
 - B = # memory blocks/pages available
 - $B(R)$ = # of bytes in each R tuple
 - $V(R, A)$ = # distinct values in R for attribute A
 - $HT(i)$ = # levels in index i
 - $LB(i)$ = # of leaf pages in index i
 - Min/Max value of attribute A in R

Extra Work

SortMergeJoin



- Sort(Employee), Sort(Works_On) on SSN
- Store merge sorted pages

HashPartitionJoin



- Hash(Employee), Hash(Works_On) on SSN
- Store merge partitioned pages

Sort Merge Join (SMJ)

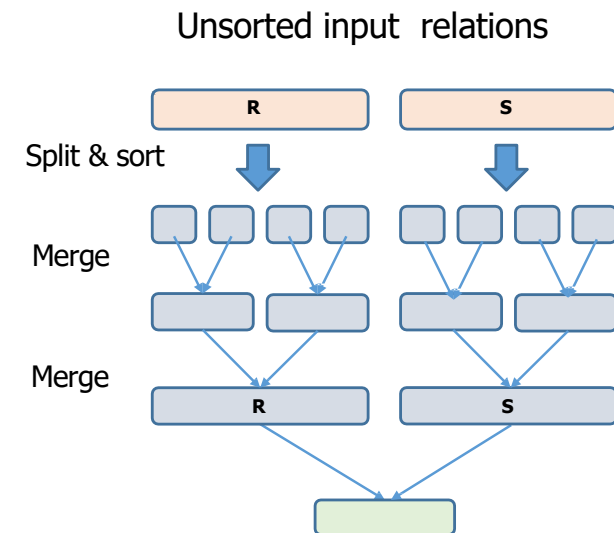
■ Goal : $R \bowtie S$ on A

■ Key idea

- Sort R and S with external sort
- Merge-scan over them

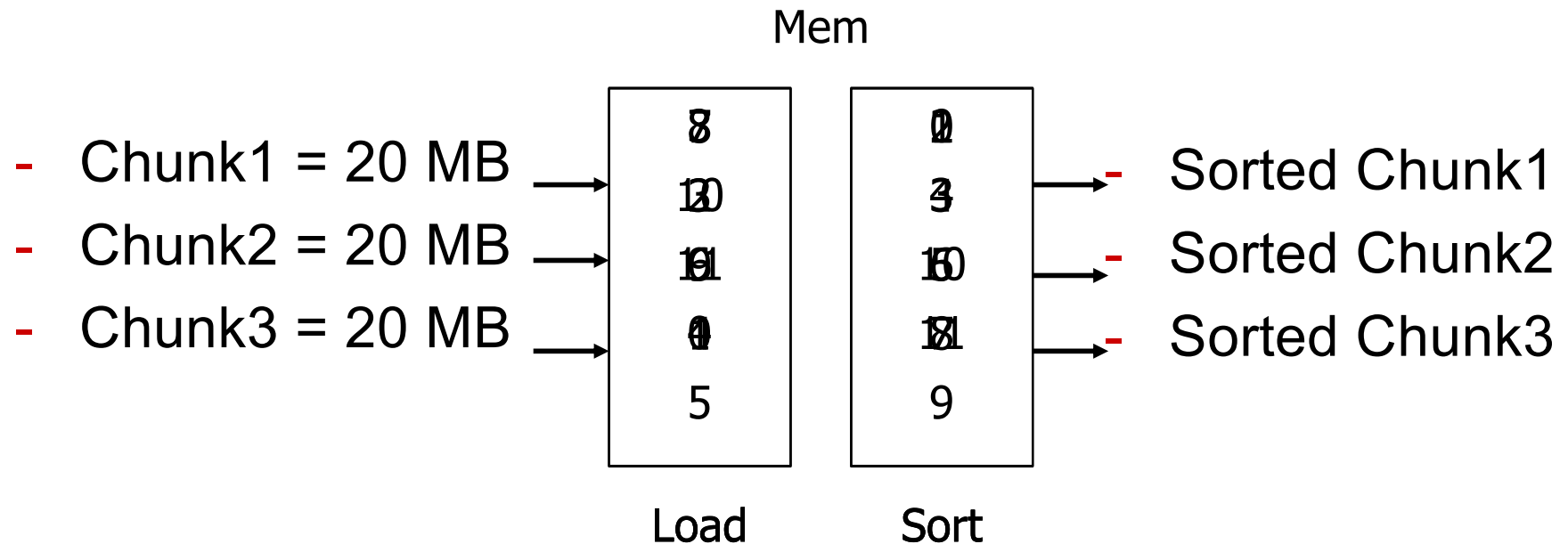
■ IO cost

- Sort phase : $sort(R) + sort(S)$
- Merge-Join phase : $\approx [P(R) + P(S) + OUT]$



BigSort (External Sort)

- $R = 60 \text{ MB}$ & $\text{Mem} = 20 \text{ MB}$
- Idea : Split $\rightarrow$ Sort $\rightarrow$ Merge



BigSort (External Sort)

- R = 60 MB & Mem = 20 MB
- Idea : Partition → Sort → Merge

Run1 : 3 5 8 8 9
Run2 : 2 4 6 7
Run3 : 0 3 10 11

0 1 2 3 3 4 5 6 7 8 9 10 11

SMJ Example

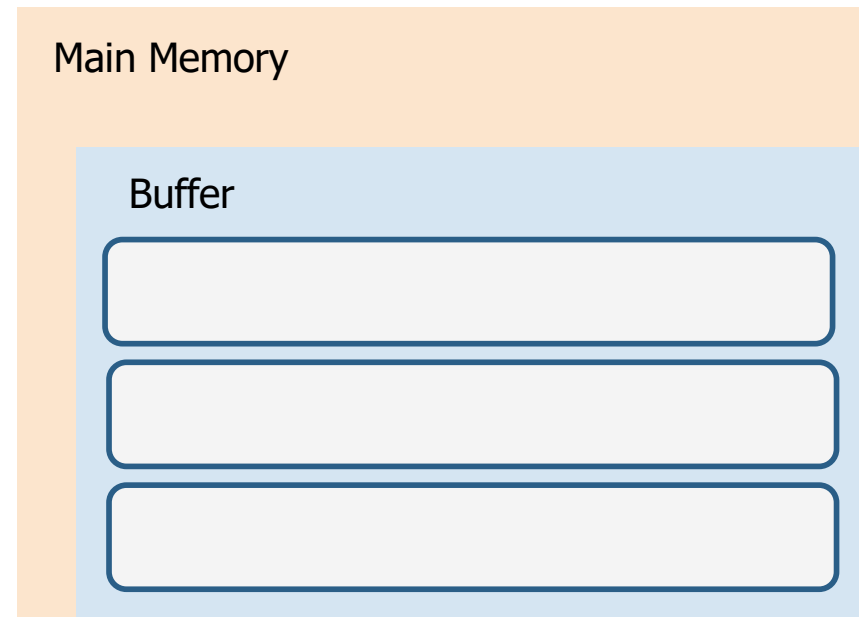
Employee, #pages=3, 6 tuples/page

```
[(432-567-789, 'S', ...), (134-562-184, 'A', ...), (613-416-452, 'J', ...), (111-222-333, 'M', ...), (000-123-456, 'C', ...), (321-777-111, 'R', ...)], [(432-567-788, 'B', ...), (222-562-184, 'Q', ...), (987-654-321, 'D', ...), (123-987-198, 'P', ...), (324-324-324, 'H', ...), (456-789-321, 'V', ...)], [(234-321-321, 'Z', ...), (453-378-123, 'T', ...), (890-764-489, 'K', ...), (924-467-184, 'K', ...), (671-268-975, 'G', ...), (512-851-586, 'W', ...)]
```

Works_On, #pages=4, 8 tuples/page

```
[(432-567-789, 'X', ...), (134-562-184, 'X', ...), (613-416-452, 'Y', ...), (111-222-333, 'Z', ...), (000-123-456, 'T', ...), (321-777-111, 'U', ...), (432-567-788, 'Y', ...), (222-562-184, 'X', ...)], [(987-654-321, 'Y', ...), (123-987-198, 'Y', ...), (324-324-324, 'X', ...), (456-789-321, 'T', ...), (234-321-321, 'U', ...), (453-378-123, 'Y', ...), (890-764-489, 'X', ...), (924-467-184, 'U', ...)], [(671-268-975, 'X', ...), (512-851-586, 'Y', ...), (432-567-789, 'Y', ...), (324-324-324, 'Y', ...), (890-764-489, 'U', ...), (222-562-184, 'U', ...), (987-654-321, 'Z', ...), (234-321-321, 'X', ...)], [(671-268-975, 'Z', ...), (512-851-586, 'U', ...), (432-567-789, 'V', ...), (321-777-111, 'V', ...), (000-123-456, 'V', ...), (222-562-184, 'Y', ...), (924-467-184, 'X', ...), (111-222-333, 'Y', ...)]
```

3 page buffer



SMJ Example

Sorted SSN

Employee, #pages=3, 6 tuples/page

```
[(000-123-456, 'C', ...), (111-222-333, 'M', ...), (123-987-198, 'P', ...), (134-562-184, 'A', ...), (222-562-184, 'Q', ...), (234-321-321, 'Z', ...)], [(321-777-111, 'R', ...), (324-324-324, 'H', ...), (432-567-788, 'B', ...), (432-567-789, 'S', ...), (453-378-123, 'T', ...), (456-789-321, 'V', ...)], [(512-851-586, 'W', ...), (613-416-452, 'J', ...), (671-268-975, 'G', ...), (890-764-489, 'K', ...), (924-467-184, 'K', ...), (987-654-321, 'D', ...)]
```

Works_On, #pages=4, 8 tuples/page

```
[(000-123-456, 'T', ...), (000-123-456, 'V', ...), (111-222-333, 'Y', ...), (111-222-333, 'Z', ...), (123-987-198, 'Y', ...), (134-562-184, 'X', ...), (222-562-184, 'X', ...), (222-562-184, 'Y', ...)], [(222-562-184, 'U', ...), (234-321-321, 'X', ...), (234-321-321, 'U', ...), (321-777-111, 'U', ...), (321-777-111, 'V', ...), (324-324-324, 'X', ...), (324-324-324, 'Y', ...), (432-567-788, 'Y', ...)], [(432-567-789, 'X', ...), (432-567-789, 'Y', ...), (432-567-789, 'V', ...), (453-378-123, 'Y', ...), (456-789-321, 'T', ...), (512-851-586, 'Y', ...), (512-851-586, 'U', ...), (613-416-452, 'Y', ...)], [(671-268-975, 'X', ...), (671-268-975, 'Z', ...), (890-764-489, 'X', ...), (890-764-489, 'U', ...), (924-467-184, 'U', ...), (924-467-184, 'X', ...), (987-654-321, 'Y', ...), (987-654-321, 'Z', ...)]
```

Output

```
(000-123-456, 'C', 'T'), (000-123-456, 'C', 'V'), (111-222-333, 'M', 'Y'), (111-222-333, 'M', 'Z'), (123-987-198, 'P', 'Y'), (134-562-184, 'A', 'X'), (222-562-184, 'Q', 'X'), (222-562-184, 'Q', 'Y')
```

3 page buffer

Main Memory

Buffer

(321-777-111, 'R', ...), (324-324-324, 'H', ...), (432-567-788, 'B', ...), (432-567-789, 'S', ...), (453-378-123, 'T', ...), (456-789-321, 'V', ...)

(222-562-184, 'U', ...), (234-321-321, 'X', ...), (234-321-321, 'U', ...), (321-777-111, 'U', ...), (321-777-111, 'V', ...), (324-324-324, 'X', ...), (324-324-324, 'Y', ...), (432-567-788, 'Y', ...)

(222-562-184, 'Q', 'U'), (234-321-321, 'Z', 'X'), (234-321-321, 'Z', 'U')

Have to “backup” in the scan of Works_On and read page we’ve already read!

Backup

- At best, no backup $\rightarrow$ scan takes $P(R) + P(S)$ reads
 - If no duplicate values in join attribute
- At worst, full backup each time $\rightarrow$ scan could take $P(R) \times P(S)$ reads
 - If all duplicate values in join attribute, i.e. all tuples in R and S have the same value for the join attribute
 - Roughly, for each page of R , we'll back up and read each page of S

SMJ Summary

- Cost of sorting R and S
- Plus the cost of scanning : $\approx P(R) + P(S)$
 - Because of backup: in worst case $P(R) \times P(S)$
 - But this would be very unlikely
- Plus the cost of writing out : OUT

$$\approx \text{sort}(P(R)) + \text{sort}(P(S)) + P(R) + P(S) + OUT$$

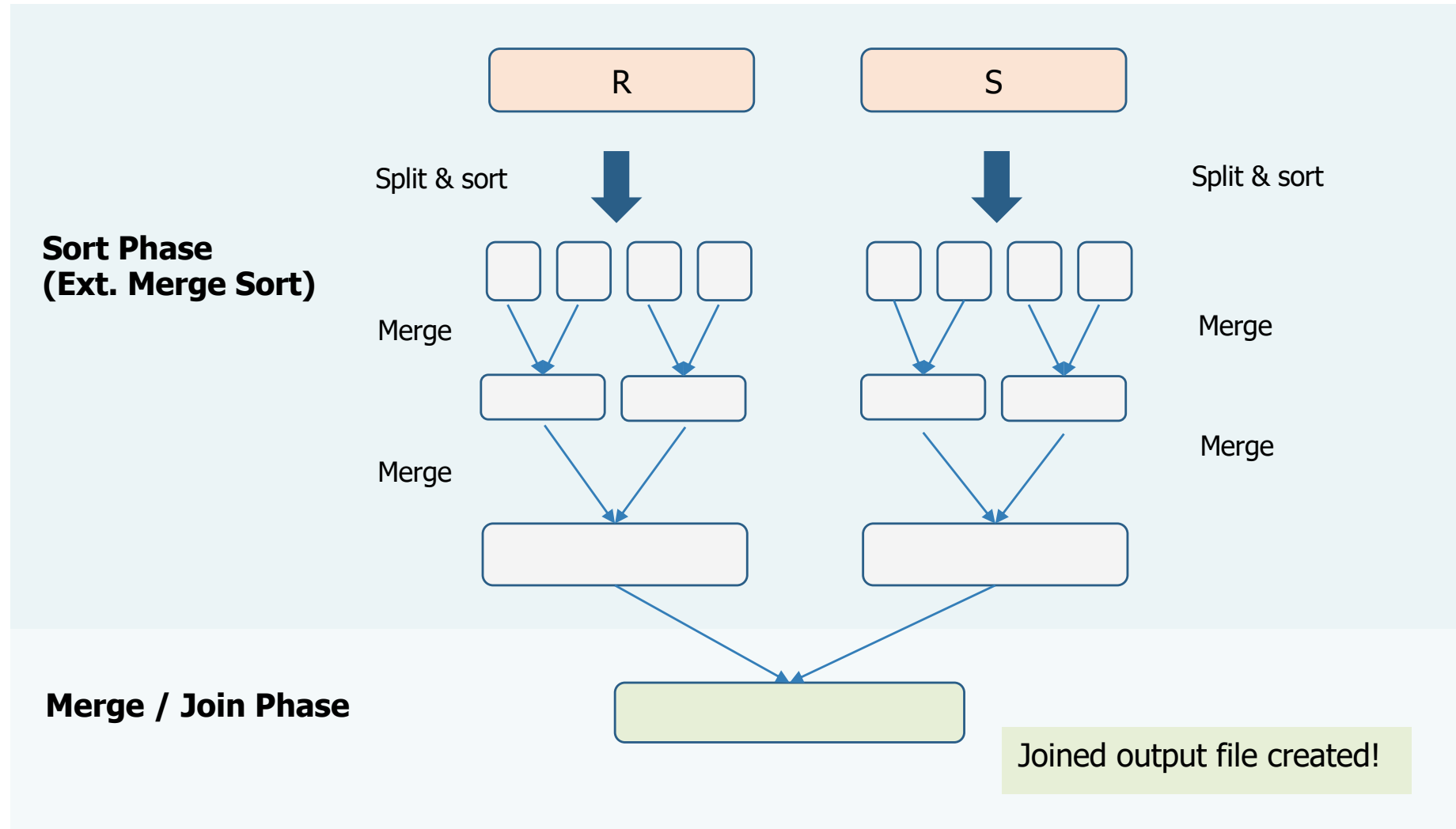
$$\text{sort}(N) \approx 2N \left(\log_B \frac{N}{2(B+1)} + 1 \right)$$

initial runs of length $\sim 2(B+1)$

Un-Optimized SMJ

Given **$B+1$** buffer pages

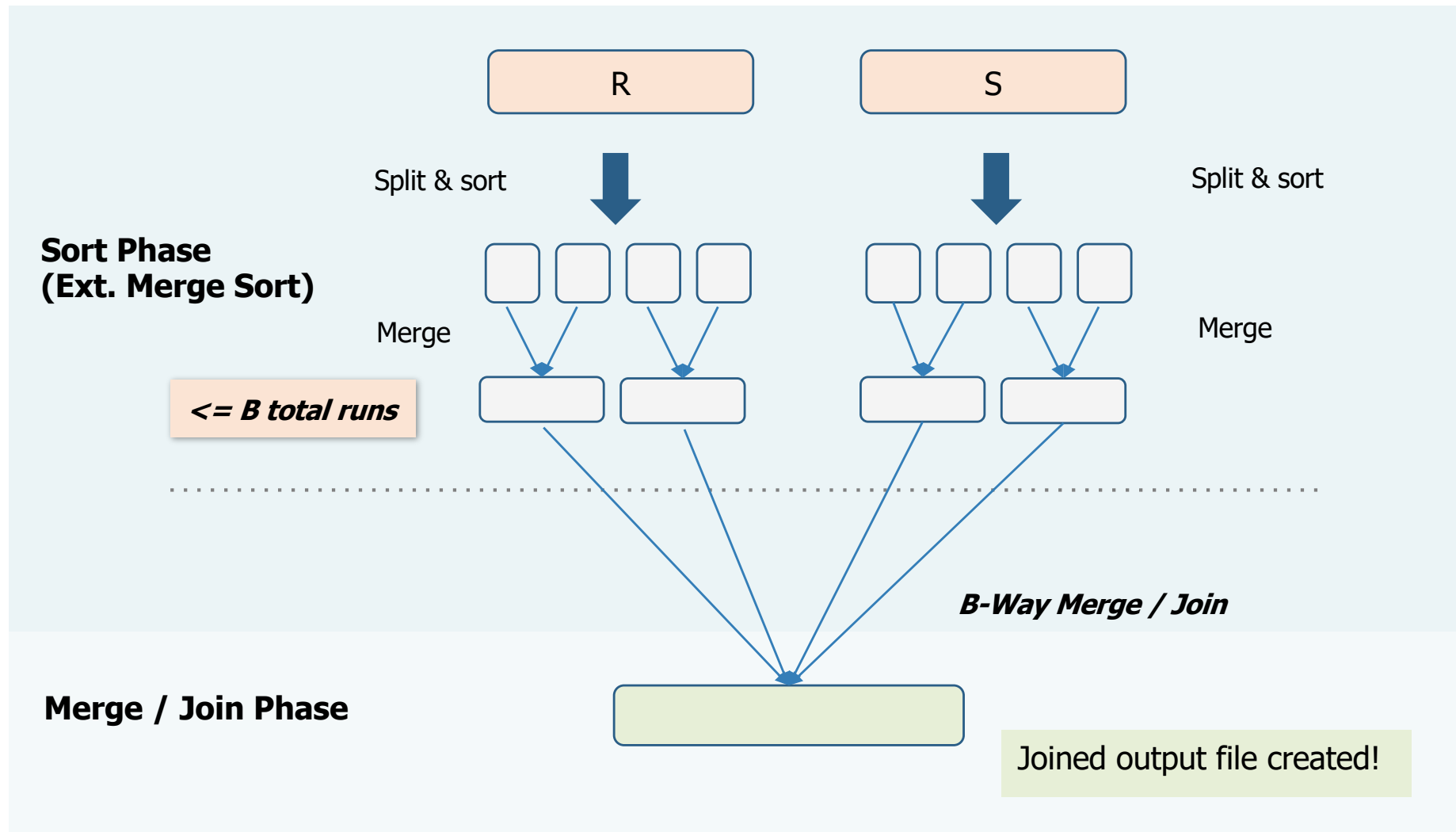
Unsorted input relations



Simple SMJ Optimization

Given **$B+1$** buffer pages

Unsorted input relations



Example

- $P(R) = 1000$, $P(S) = 500$
- 100 tuples/page
- $T(R) = 1000 \times 100$, $T(S) = 500 \times 100$

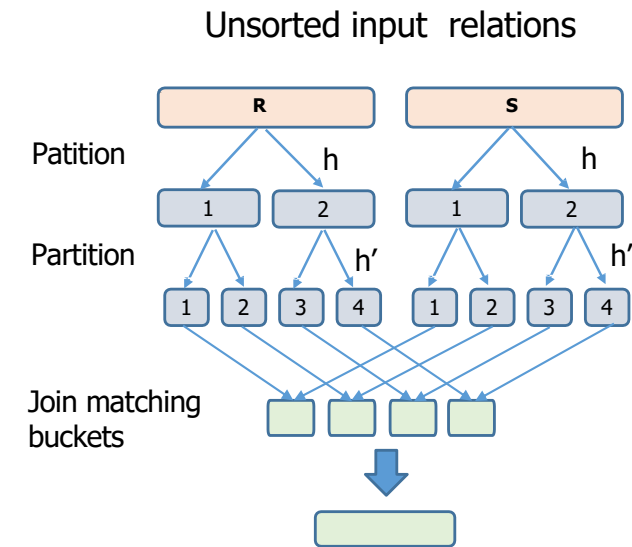
	$B + 1 = 100$ ($B = 99$)	$B + 1 = 20$ ($B = 19$)
SMJ $\text{sort}(P(R)) + \text{sort}(P(S)) + P(R) + P(S) + OUT$	<i>Sort R and S in $k = 2$ passes:</i> $2 * 1000 * k + 2 * 500 * k = 6000$ <i>Merge:</i> $1000 + 500 = 1500$ $\Rightarrow IO = 7500 + OUT$	<i>Sort R and S in $k = 3$ passes:</i> $2 * 1000 * k + 2 * 500 * k = 9000$ <i>Merge:</i> $1000 + 500 = 1500$ $\Rightarrow IO = 10,500 + OUT$
BNLJ $P(S) + \frac{P(S)}{B - 1} P(R) + OUT$	$500 + \frac{500}{99 - 1} 1000 + OUT$ $\Rightarrow IO \approx 5.6K + OUT$	$500 + \frac{500}{19 - 1} 1000 + OUT$ $\Rightarrow IO \approx 28.2K + OUT$

Hash Partition Join (HPJ)

■ Goal : $R \bowtie S$ on A

■ Key idea

- Partition R and S into buckets
 - By hashing the join attribute
- Join the pair of matching buckets



■ IO cost

- Partition phase : $partition(R) + partition(S)$
- Join phase : $\approx [P(R) + P(S) + OUT]$

Phase 1 : Hash Partitioning

■ Goal

- For each relation, partition relation into buckets such that if $h_B(t.A) = h_B(t'.A)$, they are in the same bucket

■ Given $B + 1$ buffer pages, partition into B buckets

- B buffer pages for output
- 1 page for input
 - For each tuple t in input, copy to buffer page for $h_B(t.A)$
 - When page fills up, write to disk

Phase 2 : Partition Join

- Just join pairs of buckets from R and S that have the same hash value to complete the join

HPJ Example

Partition into $B = 2$ buckets using
hash function $h_2 = \% 2$

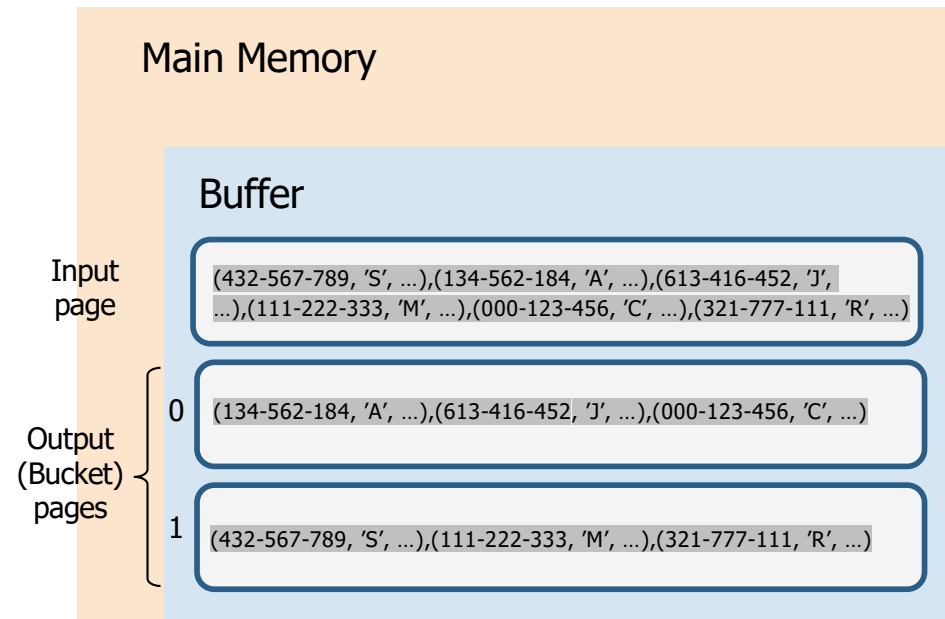
Employee, #pages=3, 6 tuples/page

```
[(432-567-789, 'S', ...), (134-562-184, 'A', ...), (613-416-452, 'J', ...), (111-222-333, 'M', ...), (000-123-456, 'C', ...), (321-777-111, 'R', ...)], [(432-567-788, 'B', ...), (222-562-184, 'Q', ...), (987-654-321, 'D', ...), (123-987-198, 'P', ...), (324-324-324, 'H', ...), (456-789-321, 'V', ...)], [(234-321-321, 'Z', ...), (453-378-123, 'T', ...), (890-764-489, 'K', ...), (924-467-184, 'K', ...), (671-268-975, 'G', ...), (512-851-586, 'W', ...)]
```

Works_On, #pages=4, 8 tuples/page

```
[(432-567-789, 'X', ...), (134-562-184, 'X', ...), (613-416-452, 'Y', ...), (111-222-333, 'Z', ...), (000-123-456, 'T', ...), (321-777-111, 'U', ...), (432-567-788, 'Y', ...), (222-562-184, 'X', ...)], [(987-654-321, 'Y', ...), (123-987-198, 'Y', ...), (324-324-324, 'X', ...), (456-789-321, 'T', ...), (234-321-321, 'U', ...), (453-378-123, 'Y', ...), (890-764-489, 'X', ...), (924-467-184, 'U', ...)], [(671-268-975, 'X', ...), (512-851-586, 'Y', ...), (432-567-789, 'Y', ...), (324-324-324, 'Y', ...), (890-764-489, 'U', ...), (222-562-184, 'U', ...), (987-654-321, 'Z', ...), (234-321-321, 'X', ...)], [(671-268-975, 'Z', ...), (512-851-586, 'U', ...), (432-567-789, 'V', ...), (321-777-111, 'V', ...), (000-123-456, 'V', ...), (222-562-184, 'Y', ...), (924-467-184, 'X', ...), (111-222-333, 'Y', ...)]
```

Given $B + 1 = 3$ buffer pages



Note that collisions can occur!

HPJ Example

Partition into $B = 2$ buckets using
hash function $h_2 = \% 2$

Given $B + 1 = 3$ buffer pages

Employee, #pages=4, 6 tuples/page

```

0 [(134-562-184, 'A', ...), (613-416-452, 'J', ...), (000-123-456, 'C',
  ...), (432-567-788, 'B', ...), (222-562-184, 'Q', ...), (123-987-198, 'P',
  ...)], [(324-324-324, 'H', ...)], [(924-467-184, 'K', ...), (512-851-586,
  'W', ...), (none, none), (none, none), (none, none)]

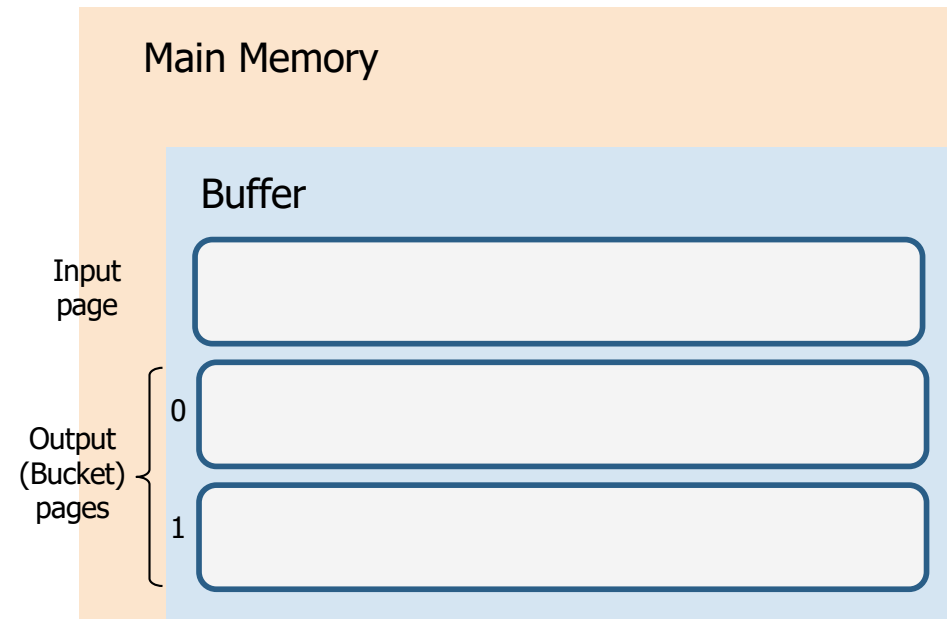
1 [(432-567-789, 'S', ...), (111-222-333, 'M', ...), (321-777-111, 'R',
  ...), (987-654-321, 'D', ...), (456-789-321, 'V', ...), (234-321-321, 'Z',
  ...)], [(453-378-123, 'T', ...), (890-764-489, 'K', ...), (671-268-975,
  'G', ...), (none, none), (none, none), (none, none)]
  
```

Works_On, #pages=5, 8 tuples/page

```

0 [(134-562-184, 'X', ...), (613-416-452, 'Y', ...), (000-123-456, 'T',
  ...), (432-567-788, 'Y', ...), (222-562-184, 'X', ...), (123-987-198, 'Y',
  ...), (324-324-324, 'X', ...), (924-467-184, 'U', ...)], [(512-851-586,
  'Y', ...), (324-324-324, 'Y', ...), (222-562-184, 'U', ...), (512-851-586,
  'U', ...), (000-123-456, 'V', ...), (222-562-184, 'Y', ...), (924-467-184,
  'X', ...), (none, none)]

1 [(432-567-789, 'X', ...), (111-222-333, 'Z', ...), (321-777-111, 'U',
  ...), (987-654-321, 'Y', ...), (456-789-321, 'T', ...), (234-321-321, 'U',
  ...), (453-378-123, 'Y', ...), (890-764-489, 'X', ...)], [(671-268-975,
  'X', ...), (432-567-789, 'Y', ...), (890-764-489, 'U', ...), (987-654-321,
  'Z', ...), (234-321-321, 'X', ...)], [(671-268-975, 'Z', ...), (432-567-
  789, 'V', ...), (321-777-111, 'V', ...)], [(111-222-333, 'Y', ...),
  (none, none), (none, none), (none, none), (none, none), (none, none),
  (none, none), (none, none)]
  
```



Unevenness in the bucket size: **skew**

HPJ Example

Partition into $B = 2$ buckets using
hash function $h_2 = \% 2$

Given $B + 1 = 3$ buffer pages

Employee, #pages=4, 6 tuples/page

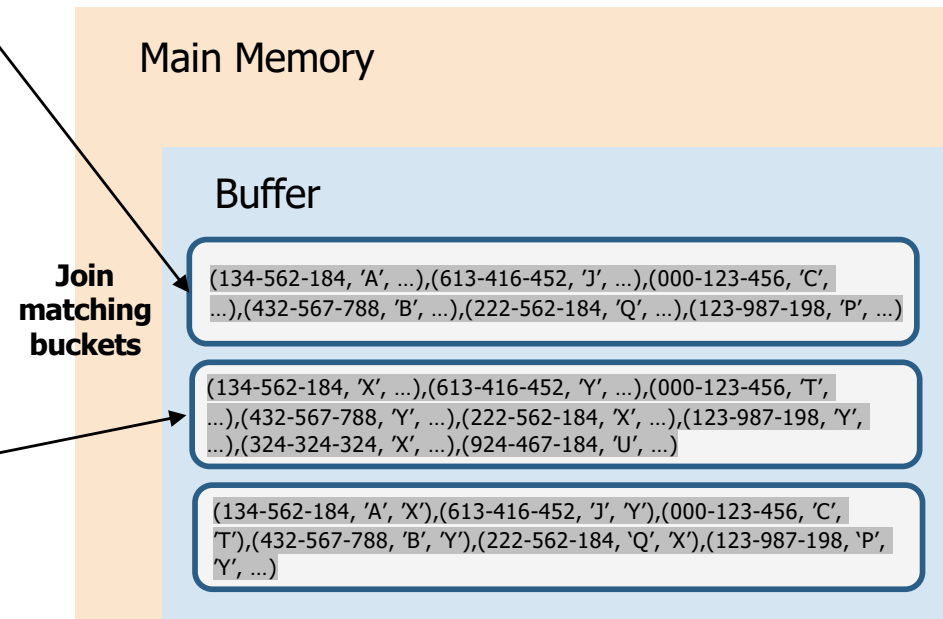
0 [(134-562-184, 'A', ...), (613-416-452, 'J', ...), (000-123-456, 'C', ...), (432-567-788, 'B', ...), (222-562-184, 'Q', ...), (123-987-198, 'P', ...), [(324-324-324, 'H', ...)], [(924-467-184, 'K', ...), (512-851-586, 'W', ...), (none, none), (none, none), (none, none)]

1 [(432-567-789, 'S', ...), (111-222-333, 'M', ...), (321-777-111, 'R', ...), (987-654-321, 'D', ...), (456-789-321, 'V', ...), (234-321-321, 'Z', ...), [(453-378-123, 'T', ...), (890-764-489, 'K', ...), (671-268-975, 'G', ...), (none, none), (none, none), (none, none)]

Works_On, #pages=5, 8 tuples/page

0 [(134-562-184, 'X', ...), (613-416-452, 'Y', ...), (000-123-456, 'T', ...), (432-567-788, 'Y', ...), (222-562-184, 'X', ...), (123-987-198, 'Y', ...), (324-324-324, 'X', ...), (924-467-184, 'U', ...)], [(512-851-586, 'Y', ...), (324-324-324, 'Y', ...), (222-562-184, 'U', ...), (512-851-586, 'U', ...), (000-123-456, 'V', ...), (222-562-184, 'Y', ...), (924-467-184, 'X', ...), (none, none)]

1 [(432-567-789, 'X', ...), (111-222-333, 'Z', ...), (321-777-111, 'U', ...), (987-654-321, 'Y', ...), (456-789-321, 'T', ...), (234-321-321, 'U', ...), (453-378-123, 'Y', ...), (890-764-489, 'X', ...)], [(671-268-975, 'X', ...), (432-567-789, 'Y', ...), (890-764-489, 'U', ...), (987-654-321, 'Z', ...), (234-321-321, 'X', ...)], [(671-268-975, 'Z', ...), (432-567-789, 'V', ...), (321-777-111, 'V', ...)], [(111-222-333, 'Y', ...), (none, none), (none, none), (none, none), (none, none), (none, none), (none, none), (none, none)]



HPJ Summary

- Given enough buffer pages
 - Hash partition
 - Requires reading + writing each page of R, S
 - $\rightarrow 2[P(R) + P(S)]$ IO cost
 - Join partition
 - With BNLJ, requires reading each page of R, S
 - $\rightarrow P(R) + P(S)$ IO cost

$$\approx 3 [P(R) + P(S)] + OUT$$

SMJ vs. HPJ

- Given enough buffer pages

$$\approx 3 [P(R) + P(S)] + OUT$$

- For smaller RAM and “worse” data distribution
 - $\approx k * [P(R) + P(S)] + OUT$, for ‘small’ $k = 3...5$

- HPJ is highly parallelizable
- SMJ is less sensitive to data skew and result is sorted

